Programmentwurf ASE

Umfang und Bewertung

Programmentwurf

„Ein Programmentwurf umfasst die Bearbeitung einer Aufgabenstellung mit der Auswahl geeigneter Methoden, der Formulierung der verwendeten Algorithmen in einer Programmiersprache, das Testen und Überprüfen der Ergebnisse auf Richtigkeit und die Programmdokumentation.“

https://www.dhbw.de/fileadmin/user_upload/Dokumente/Amtliche_Bekanntmachungen/2017/20_2017_Bekanntmachung_StuPrO_DHBW_Technik.pdf

AUFGABEN

(S): Schriftlich (Teil der Dokumentation)
(P): Programmieren (Teil des Codes)

1 Domain Driven Design
- (S) Analyse der Ubiquitous Language
- (P) Verwendung taktischer Muster des DDD
  - Value Objects
  - Entities
  - Aggregates
  - Repositories
  - Domain Services
- (S) Analyse und Begründung der verwendeten Muster

2 Clean Architecture
- (S) Schichtarchitektur planen und begründen
- (P) mindestens zwei Schichten umsetzen

Praxisprojekt
- Code und schriftliche Dokumentation
- Objektorientierte JVM-Programmiersprache (bswp. Java, Kotlin, Groovy)
- >= 1500 Zeilen Code
- >= 20 Klassen
- Ausführbar, testbar
- Persistenzschicht
- Klar definierter (sinnvoller) Nutzen
- Desktop- oder Webanwendung

3 Programming Principles
- (S) Analyse und Begründung für 5 (fünf) der vorgestellten Prinzipien aus
  - SOLID
  - GRASP
  - DRY
  - …

4 Unit Tests
- (P) mindestens 10 Unit Tests
  - Beachtung der ATRIP-Regeln
- (P) Einsatz von Mocks

5 Refactoring
- (S) Code Smells identifizieren
- Mindestens 2 Refactorings
  - (P) anwenden bzw. durchführen und
  - (S) begründen

6 Entwurfsmuster
- (P) mindestens ein Entwurfsmuster einsetzen
- (S) Einsatz ausführlich begründen

Artefakte

Die Abgabe erfolgt ausschließlich über Moodle.

Für die Abgabe sind folgende Artefakte einzureichen:

1. Schriftliche Dokumentation
   - Dateiname: “TINF<Jahr>B4_ASE_<Nachname>_<Vorname>.pdf”
   - Beispiel: “TINF21B4_ASE_Muster_Max.pdf”
2. Source-Code als git-Repository inkl. aller commits in einer der folgenden Formen:
   - Link zu Repository als Texteingabe bei Abgabe in Moodle
   - Datei mit Link zu Repository
   - .zip/.tgz-Archiv des Repositories

Anforderungen an Source Code (P)

Alle mit (P) gekennzeichneten Aufgaben liegen im Quelltext vor und genügen diesen Anforderungen:

- Objektorientierte JVM-Programmiersprache (bswp. Java, Kotlin, Groovy)
- Versionskontrolle (git)
- Vollständiges Repository abgeben (hochladen)
- Vollständiger Source Code
- Kompilierbar
- Testbar
- Ausführbar
- „Works on any (reasonable) machine“
- **Fremd-Bibliotheken als Teil des Projekts erlaubt**

Anforderungen an schriftliche Dokumentation (S)

Alle mit (S) gekennzeichneten Aufgaben genügen folgenden Anforderungen:

- Keine Mindest- und Maximalseitenzahl
- Kein akademischer Text
- Keine Benutzerdokumentation, sondern technische Dokumentation
- Enthält Ergebnisse der Analysen und **Begründungen** der Entscheidungen
- Gliederung nach Aufgabe (1 DDD, 2 Clean Architecture, …)
- Code-Strukturen ggf. als UML-Diagramme notiert, sofern hilfreich
- Detailgrad an die Aussage anpassen
- **Aussagen und Begründungen sind – wo immer möglich – durch den Verweis auf Beispiele aus dem Source Code des Projektes zu belegen!**

Die Dokumentation ist als einzelnes PDF-Dokument abzugeben mit eigenem Kapitel/Sektion/… pro Aufgabe.

Gruppenarbeit

- Ist grundsätzlich möglich
- Gruppengröße maximal zwei Personen
- Die Leistung jedes einzelnen Team-Mitglieds muss **eindeutig identifizierbar** sein
  - ~ 1500 **zuordenbare** Zeilen Code pro Teammitglied!
  - Eigene Dokumentation pro Teammitglied!
  - Rechnen Sie mit Fragen
- **Wird nicht empfohlen**

Bewertung

- Die Bewertung erfolgt anhand des Source Codes und der schriftlichen Dokumentation
- Aussagen und Begründungen sind – wo immer möglich – durch den Verweis auf Beispiele aus dem Source Code des Projektes zu belegen!

Sprechstunde

- Im sechsten Semester wird eine regelmäßige, wöchentliche Sprechstunde angeboten, um Fragen zu besprechen und Entwürfe zu diskutieren
- **Teilnahme wird empfohlen**

Weiteres Vorgehen

- Mitteilung der Themen bis spätestens **06.12.2023** per E-Mail
- Kurze Vorstellung und Beschreibung des Projektes
- **Die Vorlage in Moodle ist zu verwenden**
- Dateiname “TINF<Jahr>B4_Thema_PE_<Nachname>_<Vorname>.pdf“
  - Beispiel: „TINF21B4_Thema_PE_Muster_Max.pdf“

Themenauswahl

- Thema mit hinreichend komplexer, fachlicher Problemdomäne

  Gut:

  - RAPLA
  - Finanzplaner
  - Kontaktverwaltung

  Nicht erlaubt:

  - Spiele
  - Rein technische Themen (Algorithmen, Engines, …)
  - Mobile Apps

Themenvorschläge aus den letzten Semestern

- Aufgabenverwaltung / ToDo-Liste
- Barbestands-Plane (Getränkebestand verwalten, auswerten und abrechnen)
- Benutzer-Verwaltung
- Bibliothek-Verwaltung
- Einkaufslisten-Planer
- Finanzmanager /Budget-Planer / Einkommen vs. Ausgaben
- Gain-Watchers (Trainings-Anwendung mit Muskel-Zugewinn)
- Musikverein-Verwaltung mit Instrumenten-Ausleihe
- WG-Organizer
- Wohnheim-Verwaltung

Details zu den Aufgaben

Domain-Driven Design

- Analyse der Ubiquitous Language
  - Analysieren Sie die Fachlichkeit Ihrer Problemdomäne, indem Sie die relevanten Begriffe und deren fachliche Bedeutung, Aufgaben und Regeln dokumentieren
- Verwendung taktischer Muster des DDD
  - **Alle** genannten Muster des DDD sind im Source Code zu verwenden (Value Objects, Entities, Aggregates, Repositories, Domain Services)
- Analyse und Begründung der verwendeten Muster
  - **Begründen** Sie jedes einzelne, oben genannte Muster anhand von je einem konkreten Beispiel aus Ihrem Source Code
  - „XX ist als ValueObject implementiert, weil…“

Clean Architecture

- Schichtarchitektur planen und begründen
  - Planen und begründen Sie eine Architektur für Ihre Anwendung, die sich an den Prinzipien der Clean Architecture orientiert und dokumentieren Sie diese
    - Welche Schichten umfasst ihre Anwendung und warum
    - Welche Aufgaben erfüllt welche Schicht
- mindestens zwei Schichten umsetzen
  - Setzen Sie mindestens zwei der geplanten Schichten im Source Code um

Programming Principles

- Analyse und Begründung für SOLID, GRASP, DRY
  - Begründen Sie für 5 (FÜNF) der vorgestellten Prinzipien aus SOLID, GRASP, DRY, … WO das jeweilige Prinzip in Ihrem Projekt berücksichtigt wird bzw. angewendet wird und nennen Sie ein Beispiel AUS IHREM SOURCE CODE dazu:
    - „Das Single-Responsibility-Principle besagt, … Dies wird zum Beispiel in der Klasse XX berücksichtigt, **weil**…“

Unit Tests

- Schreiben Sie mindestens 10 einzelne Unit Tests unter Berücksichtigung der ATRIP-Regeln
- Verwenden Sie bei mindestens einem Test Mocks, gerne auch unter Zuhilfename einer Mock-Bibliothek

Refactoring

- Identifizieren, nennen und begründen Sie mindestens 4 Code Smells, die
  - aktuell noch im Code existieren **oder**
  - die Sie im Laufe der Entwicklung identifiziert und beseitigt haben (dann mit Verweis auf entsprechenden commit)
  - Orientieren Sie sich an den in der Vorlesung genannten Smells, gerne auch andere Smells, zum Beispiel **https://refactoring.guru/refactoring/smells**
    Achtung: vermeiden Sie "unused import", "unused variable", "parameter could be final" und **ähnliche, "schwache" Smells**
- Nennen und **begründen** Sie **zwei konkrete** Refactorings, die Sie entweder
  - im Laufe des Projektes ausgeführt haben oder
  - die Sie zur Verbesserung des Projektes am aktuellen Code durchführen würden.
  - Orientieren Sie sich an den in der Vorlesung genannten Refactorings. Gerne dürfen Sie auch andere Refactorings aus seriösen Quellen verwenden,
    zum Beispiel **https://refactoring.guru/refactoring/techniques**

Entwurfsmuster

- Setzen Sie mindestens ein konkretes Entwurfsmuster im Code um
  - Nicht erlaubt: alle DDD-Muster, Singleton
- **Begründen Sie den Einsatz dieses Entwurfsmusters**
  - Warum setzen Sie dieses Muster an dieser Stelle ein
  - Wie verbessert das Muster den Code
  - Welche Vorteile/Nachteile gib es durch den Einsatz dieses Musters
  - Welche Vorteile/Nachteile gäbe es ohne dieses Muster
  - …